1. What is Kubernetes?

Kubernetes is a container orchestration platform used to deploy, manage, scale, and maintain containerized applications.
Docker is mainly used to build images and run containers, while Kubernetes is used when we have many containers and need a system to manage them automatically.
Suppose our application is running in production and we have many containers.
We need to manage things such as:
Container deployment
Container scaling
Load balancing
Service discovery
Container failures
Rolling updates
Self-healing
This is where Kubernetes is used.
EXAMPLE:
Let's understand Kubernetes completely from the basics, without assuming much Kubernetes knowledge. Let's take just one example.
Suppose we created an e-commerce application.
That application has:
Frontend → User interface
Backend → Application logic
Database → Stores application data
Suppose the backend application is very busy because thousands of users are accessing the website.
If we are using Docker manually, we may need to create and manage multiple containers ourselves.
For example:
Backend Container 1
Backend Container 2
Backend Container 3
Backend Container 4
Now imagine one backend container crashes.
We need a system that can detect the failure and create another container automatically.
Kubernetes does this kind of orchestration.
We tell Kubernetes:
"I need 4 copies of my backend application running."
Kubernetes tries to maintain that desired state.
For example:
Backend → 4 replicas required
If one replica fails:
4 replicas → One fails → Kubernetes creates/replaces a replica → 4 replicas maintained
Kubernetes can also distribute traffic between application instances.
For example:
User Requests
↓
Kubernetes Service
↓
Backend Container 1
Backend Container 2
Backend Container 3
Backend Container 4
So users do not need to know which individual container is running their request.
Kubernetes can also scale the application.
Suppose normally we need 4 backend replicas.
During heavy traffic, we may increase it to:
4 replicas → 8 replicas
When traffic becomes lower:
8 replicas → 4 replicas
Kubernetes can also perform rolling updates.
Suppose version 1 of our application is currently running:
Backend v1 → 4 replicas
Now we want to deploy version 2.
Kubernetes can gradually replace the old version with the new version instead of stopping the entire application at once.
Kubernetes also provides self-healing behavior.
Suppose one container crashes:
Container 1 → Running
Container 2 → Running
Container 3 → Crashed
Container 4 → Running
Kubernetes detects that the desired number of replicas is no longer available and works to restore the desired state.
Simple flow:
Container Image → Kubernetes → Pods → Containers → Application Running

Main point:
Docker = Build and run containers.
Kubernetes = Manage and orchestrate containers at scale.
👉 Docker = Container platform
👉 Kubernetes = Container orchestration platform
;;
2. What is a Kubernetes Cluster?
A Kubernetes Cluster is a group of machines called nodes that work together to run and manage containerized applications.
A Kubernetes cluster mainly contains:
Control Plane → Manages the Kubernetes cluster.
Worker Nodes → Run the application workloads.
The Control Plane decides what should happen in the cluster, while worker nodes run the actual workloads.
EXAMPLE:
Suppose we have an e-commerce application.
We create a Kubernetes cluster with:
Control Plane
Worker Node 1
Worker Node 2
Worker Node 3
The Control Plane manages the cluster.
The worker nodes run the application workloads.
For example:
Control Plane
↓
Worker Node 1 → Backend Pod
Worker Node 2 → Backend Pod
Worker Node 3 → Backend Pod
Suppose Worker Node 2 has a problem.
Kubernetes can detect the node problem and reschedule workloads onto available nodes, depending on the workload configuration and cluster capacity.
So the application does not have to depend on only one machine.
Simple flow:
Kubernetes Cluster
↓
Control Plane
↓
Worker Nodes
↓
Pods
↓
Containers
Main point:
Kubernetes Cluster = A group of nodes managed by Kubernetes to run containerized applications.
;;
3. What is Kubernetes Control Plane?
The Kubernetes Control Plane is the management layer of a Kubernetes cluster.
It makes decisions about the cluster and maintains the desired state of applications.
The Control Plane contains important Kubernetes components such as:
API Server
Scheduler
Controller Manager
etcd
EXAMPLE:
Suppose we tell Kubernetes:
“I want 3 replicas of my backend application.”
The request goes through the Kubernetes API Server.
The Kubernetes control components store and process the desired state.
The Scheduler decides which worker node should run a newly created Pod.
The Controllers continuously compare the actual state with the desired state and take actions when something needs to change.
The cluster state is stored in etcd.
Simple flow:
User / kubectl
↓
API Server
↓
Control Plane
↓
Worker Nodes
↓
Pods
Main point:

Control Plane = Brain/management layer of the Kubernetes cluster.
;;
4. What is Kubernetes Worker Node?
A Worker Node is a machine in a Kubernetes cluster where application workloads run.
A worker node can run multiple Pods.
The main components commonly associated with a worker node are:
Kubelet
Container Runtime
Kube-proxy
EXAMPLE:
Suppose our Kubernetes cluster has:
Worker Node 1
Worker Node 2
Worker Node 3
Our backend application needs 3 replicas.
Kubernetes can schedule:
Worker Node 1 → Backend Pod
Worker Node 2 → Backend Pod
Worker Node 3 → Backend Pod
The containers for these workloads run through the container runtime.
The Kubelet communicates with the Kubernetes control plane and makes sure the Pods assigned to that node are running as specified.
Simple flow:
Control Plane → Worker Node → Pod → Container → Application
Main point:
Worker Node = Machine that runs Kubernetes application workloads.
;;
5. What is a Kubernetes Pod?
A Pod is the smallest deployable unit in Kubernetes.
A Pod can contain one or more containers that share the same network namespace and can share storage volumes.
In most common application deployments, one Pod contains one main application container.
EXAMPLE:
Suppose we have a backend application.
We create a Pod for it.
Backend Pod
↓
Backend Container
The application container runs inside the Pod.
Now suppose we need 3 copies of the backend:
Pod 1 → Backend Container
Pod 2 → Backend Container
Pod 3 → Backend Container
These are three separate Pods.
Kubernetes generally manages Pods rather than directly managing individual containers.
Pods can also contain multiple tightly coupled containers.
For example:
Pod
↓
Main Application Container
Logging/Helper Container
These containers share the Pod's network namespace and can communicate with each other through localhost.
Simple flow:
Kubernetes → Pod → Container → Application
Main point:
Pod = Smallest deployable unit in Kubernetes and the execution environment for one or more containers.
;;
6. What is Kubernetes Deployment?
A Kubernetes Deployment is a Kubernetes resource used to manage a set of replicated Pods and control how application updates are performed.
Suppose we have a backend application.
We want:
3 replicas
We can define a Deployment that specifies the desired number of replicas and the container image to run.
The Deployment creates and manages ReplicaSets, which in turn maintain the required Pods.
EXAMPLE:
Suppose our Deployment says:
Backend replicas = 3
Kubernetes maintains:
Pod 1 → Backend
Pod 2 → Backend
Pod 3 → Backend

If one Pod fails, the Deployment's ReplicaSet works with Kubernetes to create a replacement so the desired number of replicas can be maintained.
Now suppose we release:
Backend v2
The Deployment can perform a rolling update, gradually replacing Pods running the old version with Pods running the new version.
We can check Deployments using:
kubectl get deployments
Simple flow:
Deployment → ReplicaSet → Pods → Containers
Main point:
Deployment = Manages replicated Pods and application updates.
;;
7. What is Kubernetes ReplicaSet?
A ReplicaSet is a Kubernetes resource that ensures a specified number of identical Pods are running.
Suppose we want:
3 backend Pods
The ReplicaSet tries to maintain:
Backend Pod 1
Backend Pod 2
Backend Pod 3
If one Pod is deleted:
3 Pods → One Pod deleted → ReplicaSet creates a replacement → 3 Pods
EXAMPLE:
Suppose our Deployment specifies:
replicas: 3
The Deployment creates or manages a ReplicaSet.
The ReplicaSet then maintains the desired number of Pods.
The ReplicaSet itself is usually managed by a Deployment rather than created directly for normal application deployments.
Simple flow:
Deployment → ReplicaSet → 3 Pods → Containers
Main point:
ReplicaSet = Maintains the desired number of identical Pods.
;;
8. What is Kubernetes Service?
A Kubernetes Service provides a stable network endpoint for accessing a group of Pods.
Pods are temporary and their IP addresses can change when Pods are recreated.
If users directly connect to Pod IP addresses, the application becomes difficult to manage.
A Service solves this problem by providing a stable endpoint and routing traffic to matching Pods.
EXAMPLE:
Suppose we have:
Backend Pod 1
Backend Pod 2
Backend Pod 3
The Pods may have changing IP addresses.
Instead of users connecting directly to those Pod IPs, we create:
backend-service
The Service selects the backend Pods and provides a stable way to access them.
Flow:
User Request
↓
Backend Service
↓
Backend Pod 1
Backend Pod 2
Backend Pod 3
If Pod 2 is removed and a new Pod is created, the Service can continue directing traffic to the available matching Pods.
Main point:
Service = Stable network endpoint for accessing Kubernetes Pods.
;;
9. What is Kubernetes Namespace?
A Kubernetes Namespace is used to logically separate and organize resources inside a Kubernetes cluster.
Suppose one Kubernetes cluster is used by different teams or environments.
We can create namespaces such as:
development
testing
production
Now resources can be organized separately.
EXAMPLE:
Development namespace:
Backend Deployment
Frontend Deployment

Database resources
Production namespace:
Backend Deployment
Frontend Deployment
Database resources
Both environments can exist inside the same Kubernetes cluster while being logically separated.
We can create a namespace:
kubectl create namespace development
We can see namespaces:
kubectl get namespaces
We can list Pods in a particular namespace:
kubectl get pods -n development
Namespaces help organize resources and can also be used with policies, quotas, and access controls.
Main point:
Namespace = Logical separation and organization of Kubernetes resources.
;;

10. What is Kubernetes ConfigMap?

A ConfigMap is a Kubernetes object used to store non-sensitive configuration data separately from application container images.
Suppose our application needs configuration such as:
Application mode
Database host
Application URL
Feature settings
Instead of hard-coding these values into the image, we can store them in a ConfigMap.
EXAMPLE:
Suppose we create:
app-config
with:
APP_MODE=production
DB_HOST=database
A Pod can use these values as environment variables or mounted configuration files.
Now the same application image can be used in different environments.
Development:
APP_MODE=development
Production:
APP_MODE=production
We do not need to build a different Docker image only because configuration values are different.
Important:
ConfigMap is intended for non-sensitive configuration. Passwords and other sensitive values should generally be handled using Kubernetes Secrets.
Main point:
ConfigMap = Stores non-sensitive application configuration separately from the image.
;;

11. What is Kubernetes Secret?

A Kubernetes Secret is used to store sensitive information such as passwords, tokens, and credentials.
Suppose our backend application needs a database password.
We should not hard-code the password inside the Docker image.
Instead, we can create a Kubernetes Secret.
EXAMPLE:
Suppose we have:
db-password
The Pod can receive the secret as an environment variable or through a mounted file, depending on how the workload is configured.
The application can then use the password without putting it directly inside the container image.
Secrets are commonly used for:
Database passwords
API tokens
Credentials
TLS certificates
Private keys
Kubernetes Secrets provide a mechanism for handling sensitive data, but proper cluster security, access control, and encryption-at-rest configuration are still important.
Main point:
Kubernetes Secret = Kubernetes object used to provide sensitive configuration data to workloads.
;;

12. What is Kubernetes ConfigMap vs Secret?

ConfigMap and Secret are both Kubernetes objects used to provide configuration to applications, but they are intended for different types of data.
ConfigMap is used for non-sensitive configuration.
Secret is used for sensitive information.
EXAMPLE:
Suppose our application needs:

APP_ENV=production
DB_HOST=database
These are normal configuration values, so they can be stored in a ConfigMap.
Now suppose the application needs:
DB_PASSWORD=MyPassword
This is sensitive information, so it should be stored in a Secret.
Simple comparison:
ConfigMap → Non-sensitive configuration
Secret → Sensitive configuration
Both can be provided to Pods as environment variables or mounted as files.
Main point:
ConfigMap = Normal application configuration.
Secret = Sensitive application information.
;;;
13. What is Kubernetes Ingress?
Kubernetes Ingress is an API resource used to define HTTP and HTTPS routing rules for traffic entering a Kubernetes cluster.
Suppose our application has:
example.com
We want:
example.com → Frontend
example.com/api → Backend
Instead of exposing every application separately, an Ingress can define these routing rules.
EXAMPLE:
User Request:
https://example.com/
↓
Ingress
↓
Frontend Service
And:
https://example.com/api
↓
Ingress
↓
Backend Service
An Ingress resource needs an Ingress Controller to actually implement the routing behavior.
The Ingress Controller receives incoming traffic and routes it according to the configured rules.
Ingress can commonly be used for:
Host-based routing
Path-based routing
TLS termination
HTTP/HTTPS traffic management
Main point:
Ingress = Defines external HTTP/HTTPS routing rules to Kubernetes Services.
;;;
14. What is Kubernetes Ingress Controller?
An Ingress Controller is the component that implements the routing rules defined by Kubernetes Ingress resources.
The Ingress object describes what routing should happen, while the Ingress Controller performs the actual traffic handling.
EXAMPLE:
Suppose we define:
example.com → Frontend Service
example.com/api → Backend Service
The Ingress resource contains these rules.
The Ingress Controller receives the incoming request and checks the rules.
If the user requests:
example.com/
it sends the request to the Frontend Service.
If the user requests:
example.com/api
it sends the request to the Backend Service.
Simple flow:
User → Ingress Controller → Kubernetes Service → Pods
Main point:
Ingress = Routing rules.
Ingress Controller = Component that implements those routing rules.
;;;
15. What is Kubernetes StatefulSet?
A StatefulSet is a Kubernetes resource used to manage stateful applications that require stable identities, stable network names, or persistent storage associations.
Stateless applications can usually treat each Pod as interchangeable.
Stateful applications such as databases often need stable identity and storage.

EXAMPLE:
Suppose we are running a database cluster.
We create a StatefulSet with:
3 replicas.
Kubernetes can provide stable Pod names such as:
database-0
database-1
database-2
These identities remain associated with the Pods even when Pods are recreated.
StatefulSets can also work with PersistentVolumeClaims so each replica can have its own persistent storage.
For example:
database-0 → Volume 0
database-1 → Volume 1
database-2 → Volume 2
This is different from a normal stateless Deployment where Pods are generally treated as interchangeable.
Main point:
StatefulSet = Manages stateful workloads that need stable identity and persistent storage associations.
;;

16. What is Kubernetes DaemonSet?

A DaemonSet ensures that a Pod runs on each eligible node, or on each node matching its scheduling rules.
This is useful when we need one copy of a particular workload on every node.
EXAMPLE:
Suppose we need a monitoring agent on every worker node.
Our cluster has:
Worker Node 1
Worker Node 2
Worker Node 3
We create a DaemonSet for the monitoring agent.
Kubernetes runs:
Worker Node 1 → Monitoring Pod
Worker Node 2 → Monitoring Pod
Worker Node 3 → Monitoring Pod
If a new eligible worker node joins the cluster, Kubernetes can automatically create the DaemonSet Pod on that node.
If a node is removed, its DaemonSet Pod is removed with the node.
Common use cases include:
Monitoring agents
Log collection agents
Node-level networking components
Main point:
DaemonSet = Ensures a Pod runs on every eligible node.
;;

17. What is Kubernetes Job?

A Kubernetes Job is used to run a task that should complete successfully rather than run continuously.
Suppose we have a database migration task.
We do not need the migration application to run forever.
We need it to start, complete the task, and finish.
This is where a Job is useful.
EXAMPLE:
Suppose we create:
Database Migration Job
The Job starts a Pod.
The Pod runs:
database migration
After the migration completes successfully, the Pod finishes.
Kubernetes tracks the Job's completion status.
If the Pod fails, Kubernetes can create another Pod according to the Job configuration and retry the work.
Simple flow:
Job → Pod → Task Runs → Task Completes
Main point:
Job = Runs a finite task until it successfully completes.
;;

18. What is Kubernetes CronJob?

A Kubernetes CronJob is used to run Jobs automatically according to a schedule.
Suppose we want to run a database backup every day at midnight.
Instead of manually starting the Job every day, we can create a CronJob.
EXAMPLE:
Suppose our schedule is:
Every day at 12:00 AM → Database Backup
CronJob creates a Job according to the schedule.
The Job creates a Pod.
The Pod performs the backup.

Flow:
CronJob → Job → Pod → Backup Task → Complete
Another example could be:
Every hour → Clean temporary data
Every day → Generate reports
Every week → Perform maintenance task
Main point:
CronJob = Automatically creates Jobs according to a schedule.
;;
19. What is Kubernetes PersistentVolume?
A PersistentVolume, or PV, is a Kubernetes storage resource that represents storage available to the cluster.
Pods are temporary, but application data may need to survive Pod deletion or recreation.
PersistentVolume provides storage that can exist independently of a particular Pod.
EXAMPLE:
Suppose we have a database Pod.
Database Pod
↓
PersistentVolume
↓
Persistent Storage
If the database Pod is deleted and Kubernetes creates another Pod, the application can use persistent storage rather than losing its data with the old Pod.
A PV can represent storage from different backends depending on the Kubernetes environment.
Main point:
PersistentVolume = Cluster storage resource used to provide persistent storage to workloads.
;;
20. What is Kubernetes PersistentVolumeClaim?
A PersistentVolumeClaim, or PVC, is a request for storage made by a Kubernetes workload.
Instead of a Pod directly describing a particular storage device, the application can request storage through a PVC.
EXAMPLE:
Suppose our database needs:
10Gi of storage.
We create a PVC requesting:
10Gi
Kubernetes can bind that PVC to a suitable PersistentVolume.
The Pod then mounts the PVC.
Simple flow:
Pod → PVC → PersistentVolume → Storage
Suppose:
Database Pod
↓
database-pvc
↓
database-pv
↓
Persistent Storage
The application uses the PVC rather than directly managing the underlying storage resource.
Main point:
PVC = A request for persistent storage made by a workload.
;;

1. **ChatGPT said:**

ఖచ్చితంగా 👍 **21 నుంచి 60 వరకు**, మీరు ఇచ్చిన **1–20 answers style** లోనే create చేస్తాను. ఒకే **E-commerce application** example ని throughout use చేస్తాను, so concepts connect అవుతాయి.

21. What is Kubernetes StorageClass?
A StorageClass is a Kubernetes resource used to define different types of storage and allow Kubernetes to dynamically provision PersistentVolumes when applications request storage.
Suppose our application needs persistent storage.
Instead of manually creating a PersistentVolume every time, we can create a StorageClass that defines how the storage should be provisioned.
StorageClass is commonly used together with PersistentVolumeClaim.
EXAMPLE:
Suppose we are running an e-commerce application.
Our database needs:
10Gi persistent storage
We create a PVC and specify the StorageClass.
For example:
Database Pod
↓
Database PVC
↓
StorageClass

↓
PersistentVolume
↓
Persistent Storage
Suppose we have a StorageClass called:
fast-storage
The PVC can request storage using this StorageClass.
Kubernetes can then dynamically provision a suitable PersistentVolume based on the StorageClass configuration.
For example:
PVC requests 10Gi
↓
StorageClass
↓
PersistentVolume created
↓
PVC bound to PV
↓
Database Pod uses storage
StorageClass can define things such as:
Storage provisioner
Storage type
Reclaim policy
Volume binding behavior
We can create different StorageClasses for different requirements:
fast-storage
standard-storage
slow-storage
Simple flow:
Pod
↓
PVC
↓
StorageClass
↓
PersistentVolume
↓
Persistent Storage
Main point:
StorageClass = Defines how Kubernetes dynamically provisions persistent storage for applications.
;;
22. What is Kubernetes Label and Selector?
A Label is a key-value pair attached to a Kubernetes resource to identify and organize it.
A Selector is used to find Kubernetes resources based on their labels.
Labels and Selectors are very important because Kubernetes resources such as Services and Deployments use them to identify the correct Pods.
EXAMPLE:
Suppose our e-commerce application has backend Pods.
We can give them a label:
app=backend
For example:
Backend Pod 1 → app=backend
Backend Pod 2 → app=backend
Backend Pod 3 → app=backend
Now suppose we create a Service for the backend.
The Service can use a selector:
app=backend
The Service then selects all matching backend Pods.
For example:
Backend Service
↓
selector: app=backend
↓
Backend Pod 1
Backend Pod 2
Backend Pod 3
If another Pod has:
app=frontend
the backend Service will not select that Pod.
Labels can also be used to organize resources:
app=backend
environment=production

version=v1
Simple flow:
Pod
↓
Labels
↓
Selector
↓
Matching Pods
↓
Service / Deployment
Main point:
Label = Identifies and organizes Kubernetes resources.
Selector = Finds resources based on their labels.
;;
23. What is Kubernetes Annotation?
An Annotation is metadata attached to Kubernetes resources to store additional information that is not normally used for selecting or grouping resources.
Labels are mainly used for identification and selection, while annotations are commonly used to store extra configuration or information.
EXAMPLE:
Suppose our e-commerce backend Deployment needs some additional information.
We can add annotations such as:
team=backend-team
description=E-commerce backend application
An Ingress Controller or another Kubernetes component can also use annotations for additional configuration.
For example, an Ingress may use an annotation to configure specific controller behavior.
Annotations are not normally used by Services to select Pods.
For example:
Label:
app=backend

Annotation:
description=Production backend application
Simple flow:
Kubernetes Resource
↓
Metadata
↓
Annotations
↓
Additional Information / Configuration
Main point:
Annotation = Stores additional metadata or configuration information about a Kubernetes resource.
;;
24. What is Kubernetes Resource Request and Limit?
Resource Requests and Limits define how much CPU and memory a container requires and how much it is allowed to use.
A **Request** tells Kubernetes how much CPU or memory a container needs for scheduling.
A **Limit** defines the maximum amount of CPU or memory the container can use.
EXAMPLE:
Suppose our e-commerce backend container needs:
CPU request = 500m
Memory request = 512Mi

CPU limit = 1
Memory limit = 1Gi
This means Kubernetes considers 500m CPU and 512Mi memory when scheduling the Pod.
The container can use up to the configured limits.
For example:
Backend Pod
↓
Container
↓
CPU Request: 500m
Memory Request: 512Mi
CPU Limit: 1
Memory Limit: 1Gi
Suppose a worker node has limited resources.
Kubernetes Scheduler uses resource requests to determine whether the Pod can fit on that node.
If a container exceeds its memory limit, it may be terminated by the system and can show a status such as OOMKilled.
Simple flow:

Pod
↓
Container
↓
Requests → Scheduling
↓
Limits → Maximum Resource Usage
Main point:
Request = Resource amount Kubernetes uses for scheduling.
Limit = Maximum resource amount the container is allowed to use.
;;
25. What is Kubernetes Liveness Probe?
A Liveness Probe is used to check whether a container is still healthy and running correctly.
If the liveness probe repeatedly fails, Kubernetes can restart the container according to the Pod configuration.
EXAMPLE:
Suppose our e-commerce backend application is running.
The container is running, but internally the application has become stuck and is no longer responding correctly.
We configure a liveness probe.
For example:
Backend Container
↓
Liveness Probe
↓
Application healthy?
If the application is healthy:
Probe successful → Container continues running
If the application becomes unhealthy:
Probe fails
↓
Kubernetes detects the failure
↓
Container may be restarted
Liveness probes can commonly use:
HTTP requests
TCP checks
Command execution
Simple flow:
Container
↓
Liveness Probe
↓
Healthy?
↓
Yes → Continue
No → Restart according to configuration
Main point:
Liveness Probe = Checks whether the application is still healthy enough to continue running.
;;
26. What is Kubernetes Readiness Probe?
A Readiness Probe is used to determine whether a Pod is ready to receive traffic.
If the readiness probe fails, Kubernetes removes the Pod from the Service's ready endpoints, so the Pod does not receive normal Service traffic.
EXAMPLE:
Suppose our e-commerce backend Pod has just started.
The container is running, but the application is still loading configuration and connecting to the database.
The application is not ready to receive user requests yet.
The readiness probe checks the application.
Backend Pod
↓
Readiness Probe
↓
Application ready?
If the probe succeeds:
Pod Ready
↓
Service can send traffic
If the probe fails:
Pod Not Ready
↓
Service does not send normal traffic to that Pod
This is useful during application startup and temporary unavailability.

Simple flow:
Pod
↓
Readiness Probe
↓
Ready?
↓
Yes → Receive Traffic
No → Do Not Receive Traffic
Main point:
Readiness Probe = Determines whether a Pod is ready to receive traffic.
;;
27. What is Kubernetes Startup Probe?
A Startup Probe is used to determine whether a slow-starting application has successfully started.
It is useful when an application needs a long time to start and we do not want liveness or readiness checks to interfere during startup.
EXAMPLE:
Suppose our e-commerce backend application takes 2 minutes to start because it needs to load a large configuration or initialize data.
Without a Startup Probe, a liveness check might fail while the application is still starting.
We can configure a Startup Probe.
Backend Container
↓
Startup Probe
↓
Application Started?
Until the Startup Probe succeeds, Kubernetes waits according to the configured probe behavior before normal liveness/readiness probing takes effect.
Once startup succeeds:
Startup successful
↓
Liveness Probe
Readiness Probe
can operate normally.
Simple flow:
Container Starts
↓
Startup Probe
↓
Started Successfully?
↓
Yes
↓
Liveness / Readiness Probes
Main point:
Startup Probe = Checks whether a slow-starting application has successfully started.
;;
28. What are Kubernetes Service Types?
Kubernetes Service Types define how a Service can be exposed and accessed.
The common Service types are:
ClusterIP
NodePort
LoadBalancer
ExternalName
EXAMPLE:
Suppose our e-commerce application has:
Frontend
Backend
Database
The database may only need internal cluster access.
We can use:
ClusterIP
For example:
Backend Pod
↓
Backend Service
↓
ClusterIP
If we need to expose a service through a port on every node, we can use:
NodePort
If our cloud environment provides an external load balancer, we can use:

LoadBalancer
Simple flow:
Client
↓
Service
↓
Service Type
↓
Pods
Main point:
Service Type = Defines how a Kubernetes Service is exposed and accessed.
;;
29. What is Kubernetes ClusterIP?
ClusterIP is the default Kubernetes Service type.
It provides an internal IP address that can be used to access a Service from within the Kubernetes cluster.
EXAMPLE:
Suppose our e-commerce backend has three Pods:
Backend Pod 1
Backend Pod 2
Backend Pod 3
We create:
backend-service
with the default Service type:
ClusterIP
Other Pods inside the cluster can access the backend through the Service instead of directly using Pod IP addresses.
For example:
Frontend Pod
↓
backend-service
↓
Backend Pod 1
Backend Pod 2
Backend Pod 3
The Service provides a stable internal endpoint even when individual Pods are recreated.
Simple flow:
Frontend Pod
↓
ClusterIP Service
↓
Backend Pods
Main point:
ClusterIP = Provides internal cluster access to a group of Pods.
;;
30. What is Kubernetes NodePort?
NodePort is a Kubernetes Service type that exposes a Service on a specific port on each eligible node.
It allows traffic from outside the cluster to reach the Service through a node's IP address and the assigned NodePort.
EXAMPLE:
Suppose our e-commerce frontend is running inside Kubernetes.
We create a NodePort Service.
For example:
Node IP: 192.168.1.10
NodePort: 30080
A request can reach:
Node IP:30080
↓
NodePort Service
↓
Frontend Pods
Kubernetes then forwards the traffic to the appropriate Pods.
NodePort is commonly useful for simple external access or testing, although production environments often use a LoadBalancer or Ingress depending on the architecture.
Simple flow:
User
↓
Node IP:NodePort
↓
NodePort Service
↓
Pods
Main point:
NodePort = Exposes a Service on a port of each Kubernetes node.

;;
31. What is Kubernetes LoadBalancer?
LoadBalancer is a Kubernetes Service type used to expose a Service externally using a load balancer provided by the underlying infrastructure, commonly a cloud provider.
EXAMPLE:
Suppose our e-commerce application backend needs to be accessible from outside the Kubernetes cluster.
We create a Service:
type: LoadBalancer
In a supported cloud environment, Kubernetes integrates with the platform and can provision or configure an external load balancer.
The flow can be:
User
↓
External Load Balancer
↓
Kubernetes Service
↓
Backend Pods
The external load balancer provides an externally reachable endpoint and forwards traffic toward the Kubernetes Service.
Simple flow:
Internet
↓
Cloud Load Balancer
↓
Kubernetes Service
↓
Pods
Main point:
LoadBalancer = Exposes a Kubernetes Service externally through an external load balancer.
;;
32. What is Kubernetes NetworkPolicy?
A NetworkPolicy is a Kubernetes resource used to control network communication between Pods and other network endpoints.
It can define which traffic is allowed to enter or leave selected Pods.
EXAMPLE:
Suppose our e-commerce application has:
Frontend
Backend
Database
We may want:
Frontend → Backend → Database
But we do not want the Frontend to directly access the Database.
We can use NetworkPolicy rules to restrict the communication.
For example:
Frontend
↓
Backend
↓
Database
Allowed:
Frontend → Backend
Backend → Database
Restricted:
Frontend → Database
NetworkPolicy behavior depends on whether the cluster's network plugin supports and enforces NetworkPolicy.
Simple flow:
Pod
↓
NetworkPolicy
↓
Allowed / Denied Traffic
↓
Destination
Main point:
NetworkPolicy = Controls which network traffic is allowed between workloads.
;;
33. What is Kubernetes DNS?
Kubernetes DNS provides service discovery inside the cluster.
It allows applications to communicate with Kubernetes Services using DNS names instead of manually using Service IP addresses.
EXAMPLE:
Suppose our e-commerce backend Service is:
backend-service
The frontend Pod can access the backend using the Service DNS name.

For example:
Frontend Pod
↓
backend-service
↓
Backend Service
↓
Backend Pods
If the backend Pods are recreated and their IP addresses change, the frontend does not need to know those individual Pod IPs.
The Service name remains the stable way to discover the backend.
Simple flow:
Application
↓
Service DNS Name
↓
Kubernetes Service
↓
Pods
Main point:
Kubernetes DNS = Provides service discovery using DNS names inside the cluster.
;;
34. What is Kubernetes NodeSelector?
NodeSelector is a simple Kubernetes scheduling mechanism used to run a Pod on nodes that have specific labels.
EXAMPLE:
Suppose our e-commerce application has different types of worker nodes.
For example:
Node 1 → type=general
Node 2 → type=general
Node 3 → type=high-memory
Our database application needs a high-memory node.
We can label the node:
type=high-memory
Then configure the Pod with a nodeSelector:
type=high-memory
Kubernetes Scheduler will try to place the Pod on a node with that label.
Simple flow:
Pod
↓
nodeSelector
↓
Node Label
↓
Matching Node
↓
Pod Scheduled
Main point:
NodeSelector = Schedules a Pod onto nodes with matching labels.
;;
35. What is Kubernetes Node Affinity?
Node Affinity is a more flexible way of controlling which nodes a Pod can be scheduled on based on node labels.
It provides more scheduling rules than a simple NodeSelector.
EXAMPLE:
Suppose our e-commerce database should preferably run on high-memory nodes.
We can define an affinity rule based on:
node type = high-memory
Kubernetes Scheduler uses this rule when deciding where to place the Pod.
Node affinity can express different types of requirements, including rules that are required or preferred.
For example:
Database Pod
↓
Node Affinity
↓
Prefer high-memory nodes
↓
Scheduler
↓
Suitable Worker Node
Simple flow:
Pod
↓
Node Affinity Rules

↓
Scheduler
↓
Matching / Preferred Node
Main point:
Node Affinity = Controls Pod placement using flexible node-label-based scheduling rules.
;;
36. What is Kubernetes Taint and Toleration?
A Taint is applied to a node to prevent Pods from being scheduled there unless they have a matching Toleration.
A Toleration allows a Pod to tolerate a specific taint.
EXAMPLE:
Suppose we have a worker node dedicated to database workloads.
We add a taint:
workload=database:NoSchedule
Now normal Pods should not be scheduled onto this node.
Our database Pod can have a matching toleration.
Database Node
↓
Taint
↓
Database Pod
↓
Matching Toleration
↓
Pod can be scheduled
A frontend Pod without the required toleration should not be scheduled there.
Simple flow:
Node
↓
Taint
↓
Pod
↓
Matching Toleration?
↓
Yes → Can Schedule
No → Cannot Schedule
Main point:
Taint = Restricts which Pods can be scheduled on a node.
Toleration = Allows a Pod to tolerate that node restriction.
;;
37. What is Kubernetes Horizontal Pod Autoscaler?
Horizontal Pod Autoscaler, or HPA, automatically adjusts the number of Pod replicas based on resource utilization or other supported metrics.
EXAMPLE:
Suppose our e-commerce backend normally runs:
4 replicas
During a sale, thousands of users access the application.
CPU usage increases.
HPA can increase the number of replicas:
4 replicas
↓
High traffic
↓
High CPU usage
↓
HPA
↓
8 replicas
When traffic decreases, HPA can reduce the number of replicas again, according to its configuration.
8 replicas
↓
Low traffic
↓
HPA
↓
4 replicas
Simple flow:
User Traffic
↓
CPU / Memory / Metrics

↓
HPA
↓
Increase / Decrease Replicas
↓
Pods
Main point:
HPA = Automatically scales the number of Pod replicas based on metrics.
;;
38. What is Kubernetes Vertical Pod Autoscaler?
Vertical Pod Autoscaler, or VPA, automatically adjusts the CPU and memory resource requests of Pods based on their resource usage.
Instead of increasing the number of Pods like HPA, VPA adjusts the resources assigned to Pods.
EXAMPLE:
Suppose our backend Pod initially has:
CPU request = 500m
Memory request = 512Mi
Over time, the application consistently needs more memory.
VPA can recommend or apply higher resource requests according to its configuration.
For example:
500m CPU → 1 CPU
512Mi → 1Gi
This helps applications receive resource requests that better match their actual usage.
Simple flow:
Pod
↓
Resource Usage
↓
VPA
↓
Adjusted CPU / Memory Requests
↓
Pod
Main point:
VPA = Adjusts CPU and memory resources for Pods based on usage.
;;
39. What is Kubernetes Cluster Autoscaler?
Cluster Autoscaler automatically adjusts the number of worker nodes in a cluster based on resource requirements and cluster configuration.
HPA increases or decreases Pods, while Cluster Autoscaler changes the number of nodes.
EXAMPLE:
Suppose our cluster has:
Worker Node 1
Worker Node 2
Worker Node 3
During an e-commerce sale, HPA increases backend Pods.
Now the existing nodes do not have enough capacity to schedule all the new Pods.
Some Pods may remain Pending.
Cluster Autoscaler can add additional worker nodes when supported and configured.
3 Nodes
↓
More Pods
↓
Not enough capacity
↓
Cluster Autoscaler
↓
5 Nodes
When capacity is no longer needed, it can reduce nodes according to its configuration and constraints.
Simple flow:
More Pods
↓
Insufficient Node Capacity
↓
Cluster Autoscaler
↓
More Worker Nodes
Main point:
Cluster Autoscaler = Automatically adjusts the number of worker nodes based on cluster capacity requirements.
;;
40. What is Kubernetes RBAC?

RBAC stands for Role-Based Access Control.
It is used to control what users, groups, and service accounts are allowed to do with Kubernetes resources.
The main RBAC resources include:
Role
ClusterRole
RoleBinding
ClusterRoleBinding
EXAMPLE:
Suppose our e-commerce Kubernetes cluster is used by:
Developer
DevOps Engineer
Administrator
A developer may need permission to:
View Pods
View Deployments
View Services
But the developer may not need permission to delete critical production resources.
We can use RBAC to define and assign these permissions.
For example:
Developer
↓
Role
↓
Read Pods
Read Deployments
↓
RoleBinding
Simple flow:
User / ServiceAccount
↓
Role / ClusterRole
↓
RoleBinding / ClusterRoleBinding
↓
Permissions
↓
Kubernetes Resources
Main point:
RBAC = Controls who can perform which actions on Kubernetes resources.
;;
41. What is Kubernetes ServiceAccount?
A ServiceAccount provides an identity for processes running inside Kubernetes Pods.
Applications running inside Pods can use a ServiceAccount to authenticate to the Kubernetes API when they need API access.
EXAMPLE:
Suppose our e-commerce monitoring application runs inside a Pod.
The application needs permission to read certain Kubernetes resources.
We can create a ServiceAccount:
monitoring-serviceaccount
Then we can use RBAC to give this ServiceAccount specific permissions.
Monitoring Pod
↓
ServiceAccount
↓
Role / ClusterRole
↓
Permissions
↓
Kubernetes API
This allows the application to operate with controlled permissions rather than using an administrator identity.
Simple flow:
Pod
↓
ServiceAccount
↓
RBAC
↓
Kubernetes API
Main point:
ServiceAccount = Provides an identity for applications running inside Kubernetes Pods.
;;
42. What is Kubernetes SecurityContext?

SecurityContext defines security-related settings for a Pod or container.
It can control things such as the user and group under which a container runs and certain Linux security capabilities.
EXAMPLE:
Suppose our e-commerce backend container should not run as the root user.
We can configure:
runAsNonRoot
This helps reduce security risks.
For example:
Backend Pod
↓
SecurityContext
↓
Run as non-root user
↓
Container
SecurityContext can also be used for settings such as:
runAsUser
runAsGroup
runAsNonRoot
capabilities
privileged
depending on the security requirements.
Simple flow:
Pod / Container
↓
SecurityContext
↓
Security Settings
↓
Container Runs
Main point:
SecurityContext = Defines security settings for Kubernetes Pods and containers.
;;
43. What is Kubernetes Volume?
A Volume provides storage that can be mounted inside a Pod's containers.
Kubernetes supports different types of volumes for different use cases.
EXAMPLE:
Suppose our e-commerce application has two containers inside one Pod.
They need to share temporary files.
We can use:
emptyDir
For example:
Pod
↓
emptyDir Volume
↙ ↘
Container 1 Container 2
Both containers can access the shared volume.
Other volume types can provide configuration, secrets, or persistent storage.
Simple flow:
Pod
↓
Volume
↓
Container
↓
Application Data
Main point:
Volume = Provides storage that can be mounted and used by containers in a Pod.
;;
44. What is Kubernetes Init Container?
An Init Container is a container that runs before the main application containers in a Pod.
It is used to perform initialization tasks before the main application starts.
EXAMPLE:
Suppose our e-commerce backend requires some initialization before starting.
For example:
Check database availability
We can use an Init Container.
Pod
↓
Init Container

↓
Check Database
↓
Success
↓
Main Backend Container
The main application container starts only after the Init Container completes successfully.
If the Init Container fails, Kubernetes can retry it according to the Pod configuration.
Simple flow:
Pod Starts
↓
Init Container
↓
Initialization Complete
↓
Main Container
↓
Application Running
Main point:
Init Container = Performs initialization tasks before the main application containers start.
;;
45. What is Kubernetes Job?
A Job is a Kubernetes resource used to run a task that should complete successfully rather than run continuously.
EXAMPLE:
Suppose our e-commerce application needs a database migration.
We do not want the migration container to run continuously.
We create a Job:
Database Migration Job
The Job creates a Pod.
The Pod performs the migration.
After the task completes:
Job
↓
Pod
↓
Database Migration
↓
Success
↓
Completed
If the Pod fails, the Job can retry according to its configuration.
Simple flow:
Job
↓
Pod
↓
Task Runs
↓
Task Completes
Main point:
Job = Runs a finite task until it successfully completes.
;;
46. What is Kubernetes CronJob?
A CronJob is used to automatically create Jobs according to a schedule.
EXAMPLE:
Suppose our e-commerce application needs a database backup every day at midnight.
Instead of manually creating a Job every day, we create a CronJob.
Every day at 12:00 AM
↓
CronJob
↓
Job
↓
Pod
↓
Database Backup
↓
Complete
Other examples:
Every hour → Clean temporary data
Every day → Generate reports

Every week → Maintenance task
Simple flow:
CronJob
↓
Scheduled Time
↓
Job
↓
Pod
↓
Task
↓
Complete
Main point:
CronJob = Automatically creates Jobs according to a schedule.
;;
47. What is Kubernetes Helm?
Helm is a package manager for Kubernetes that helps install, configure, upgrade, and manage Kubernetes applications using reusable packages called Charts.
EXAMPLE:
Suppose our e-commerce application contains:
Frontend
Backend
Redis
Database
Ingress
ConfigMap
Secrets
Managing many YAML files manually can become difficult.
We can package Kubernetes resources into a Helm Chart.
For example:
E-commerce Helm Chart
↓
Deployment
Service
ConfigMap
Ingress
PVC
We can use Helm commands such as:
helm install
helm upgrade
helm rollback
The same Chart can be configured differently for development and production using values.
Simple flow:
Helm Chart
↓
Values
↓
Kubernetes YAML Templates
↓
Kubernetes Resources
↓
Application
Main point:
Helm = Package manager used to deploy and manage Kubernetes applications.
;;
48. What is Kubernetes CRD?
CRD stands for Custom Resource Definition.
A CRD allows us to extend the Kubernetes API by creating our own custom resource types.
EXAMPLE:
Suppose our e-commerce platform needs a custom resource called:
DatabaseCluster
We can define this resource using a CRD.
After creating the CRD, Kubernetes can understand a custom resource such as:
DatabaseCluster
A custom controller or Operator can then watch this resource and perform actions.
For example:
DatabaseCluster
↓
Custom Controller
↓

Create / Manage Database Resources
Simple flow:
CRD
↓
Custom Resource
↓
Controller / Operator
↓
Kubernetes Resources
Main point:
CRD = Extends Kubernetes with custom resource types.
;;
49. What is Kubernetes Operator?
An Operator is a software pattern that uses Kubernetes controllers and custom resources to automate the management of applications.
Operators are commonly used for complex applications such as databases and messaging systems.
EXAMPLE:
Suppose our e-commerce application uses a database cluster.
Instead of manually managing database replicas, backups, and failover, an Operator can automate some of these tasks.
For example:
Database Custom Resource
↓
Database Operator
↓
Create / Manage Database
↓
Monitor Database
↓
Take Corrective Actions
The Operator continuously watches the desired state and manages the application according to its logic.
Simple flow:
Custom Resource
↓
Operator
↓
Controller
↓
Application Resources
↓
Desired State Maintained
Main point:
Operator = Automates the deployment and management of complex applications using Kubernetes APIs and controllers.
;;
50. What is Kubernetes API Server?
The Kubernetes API Server is the central component that exposes the Kubernetes API and acts as the main communication point for Kubernetes components and clients.
When we use kubectl, our requests are sent to the API Server.
EXAMPLE:
Suppose we want to create our e-commerce Deployment.
We run:
kubectl apply -f deployment.yaml
The request goes to the API Server.
The API Server validates and processes the request and coordinates with other Kubernetes components.
kubectl
↓
API Server
↓
Kubernetes Control Plane
↓
Worker Nodes
↓
Pods
The API Server is also used by other Kubernetes components and applications that interact with the Kubernetes API.
Simple flow:
User / kubectl
↓
API Server
↓
Kubernetes API
↓
Control Plane

↓
Cluster Resources
Main point:
API Server = Central communication point for interacting with the Kubernetes cluster.
;;
51. What is Kubernetes etcd?
etcd is a distributed key-value store used by Kubernetes to store cluster state and configuration data.
The Kubernetes Control Plane uses etcd as the persistent data store for information about the cluster.
EXAMPLE:
Suppose our e-commerce cluster has:
Backend Deployment
3 replicas
Services
ConfigMaps
Secrets
Pods
Kubernetes needs to store important cluster information so that the control plane knows the desired state and other metadata.
This information is stored in etcd.
For example:
API Server
↓
etcd
↓
Cluster State
If the control plane needs to know the stored state of Kubernetes resources, it interacts through the Kubernetes API architecture with etcd.
Simple flow:
kubectl
↓
API Server
↓
etcd
↓
Cluster State
Main point:
etcd = Distributed key-value store used by Kubernetes to store cluster state.
;;
52. What is Kubernetes Scheduler?
The Kubernetes Scheduler is a control-plane component that selects a suitable worker node for Pods that do not yet have a node assigned.
EXAMPLE:
Suppose our e-commerce backend needs a new Pod.
Backend Pod
↓
Scheduler
The Scheduler checks available nodes and considers scheduling requirements such as:
CPU / Memory
Node labels
Affinity
Taints and tolerations
Other scheduling constraints
Suppose we have:
Worker Node 1 → Not enough CPU
Worker Node 2 → Suitable
Worker Node 3 → Suitable
The Scheduler selects a suitable node according to its scheduling logic and constraints.
New Pod
↓
Scheduler
↓
Worker Node 2
↓
Kubelet
↓
Container
Simple flow:
New Pod
↓
Scheduler
↓
Suitable Worker Node

↓
Kubelet
↓
Pod Running
Main point:
Scheduler = Selects a suitable worker node for a Pod.
;;;
53. What is Kubernetes Controller Manager?
The Controller Manager is a control-plane component that runs Kubernetes controllers.
Controllers continuously compare the current state of the cluster with the desired state and take actions to move the cluster toward the desired state.
EXAMPLE:
Suppose our e-commerce Deployment requires:
3 backend replicas
But currently only:
2 Pods
are available.
A controller detects that the actual state does not match the desired state.
It works with Kubernetes to create a replacement Pod.
Desired State → 3 Pods
Actual State → 2 Pods
↓
Controller
↓
Create another Pod
↓
3 Pods
Controllers are responsible for different kinds of resources and behaviors.
Simple flow:
Desired State
↓
Controller
↓
Compare Actual State
↓
Take Action
↓
Desired State Restored
Main point:
Controller Manager = Runs controllers that help maintain the desired state of the cluster.
;;;
54. What is Kubernetes Deployment Strategy?
A Deployment Strategy defines how Kubernetes replaces existing Pods when a new application version is deployed.
Common strategies include:
RollingUpdate
Recreate
EXAMPLE:
Suppose our e-commerce backend is running:
Backend v1 → 4 Pods
Now we want to deploy:
Backend v2
With a RollingUpdate strategy, Kubernetes gradually replaces v1 Pods with v2 Pods.
v1 → v1 → v1 → v1
↓
v2 → v1 → v1 → v1
↓
v2 → v2 → v1 → v1
↓
v2 → v2 → v2 → v1
↓
v2 → v2 → v2 → v2
This helps reduce application downtime during updates.
With Recreate, the existing Pods are terminated before new Pods are created.
Simple flow:
Old Version
↓
Deployment Strategy
↓
New Version
↓
Updated Pods

Main point:
Deployment Strategy = Defines how application Pods are replaced during updates.
;;
55. What is Kubernetes Rolling Update?
Rolling Update is a deployment strategy that gradually replaces old application Pods with new Pods.
It allows an application to be updated without stopping all replicas at the same time.
EXAMPLE:
Suppose our e-commerce backend has:
v1 → 4 replicas
We release:
v2
Kubernetes gradually replaces the old Pods.
4 v1 Pods
↓
1 v2 + 3 v1
↓
2 v2 + 2 v1
↓
3 v2 + 1 v1
↓
4 v2 Pods
The exact rollout behavior depends on Deployment configuration such as update parameters and readiness.
We can check the rollout using:
kubectl rollout status deployment/backend
Simple flow:
Backend v1
↓
Rolling Update
↓
Backend v2
↓
Gradually Replace Pods
↓
All Pods v2
Main point:
Rolling Update = Gradually replaces old Pods with new Pods during application deployment.
;;
56. What is Kubernetes Blue-Green Deployment?
Blue-Green Deployment is a deployment strategy where two application environments or versions are maintained, usually called Blue and Green.
One version serves production traffic while the other version is prepared and tested.
EXAMPLE:
Suppose our e-commerce application currently runs:
Blue → Backend v1
We deploy the new version:
Green → Backend v2
We can test Green before switching production traffic.
Once Green is ready, traffic can be switched from Blue to Green.
Users
↓
Service
↓
Green → Backend v2
If a rollback is needed, traffic can be switched back to Blue.
Simple flow:
Blue → v1
Green → v2
↓
Testing
↓
Switch Traffic
↓
Green → Production
Main point:
Blue-Green = Maintains two application versions and switches traffic between them.
;;
57. What is Kubernetes Canary Deployment?
Canary Deployment is a strategy where a new application version is released to a small portion of users or traffic before it is rolled out more widely.
EXAMPLE:
Suppose our e-commerce backend currently runs:

v1 → 10 Pods
We release:
v2
Instead of sending all traffic to v2, we first expose v2 to a small percentage of traffic.
For example:
90% traffic → v1
10% traffic → v2
We monitor the new version.
If everything works correctly:
50% → v2
100% → v2
If there is a problem, we can reduce or stop traffic to v2.
Simple flow:
Users
↓
Traffic Split
↙ ↘
v1 v2
90% 10%
↓
Monitor
↓
Increase v2 Traffic
Main point:
Canary Deployment = Releases a new version gradually to a small amount of traffic before full rollout.
;;
58. What is Kubernetes Monitoring?
Kubernetes Monitoring is the process of observing the health, performance, and resource usage of Kubernetes clusters and applications.
We can monitor things such as:
CPU usage
Memory usage
Pod status
Node status
Application metrics
Request rates
Errors
EXAMPLE:
Suppose our e-commerce backend is running in Kubernetes.
We want to know:
CPU usage
Memory usage
Number of Pods
Request count
Application errors
A monitoring system can collect metrics and display them through dashboards.
Common tools include:
Prometheus
Grafana
Metrics Server
For example:
Kubernetes
↓
Metrics
↓
Prometheus
↓
Grafana
↓
Monitoring Dashboard
Simple flow:
Cluster / Applications
↓
Metrics
↓
Monitoring System
↓
Dashboard / Alerts
Main point:
Kubernetes Monitoring = Observes cluster and application health, performance, and resource usage.
;;

59. What is Kubernetes Logging?
Kubernetes Logging is the process of collecting and analyzing logs generated by applications and containers running in the cluster.
EXAMPLE:
Suppose a customer places an order in our e-commerce application.
The backend may generate logs such as:
Order received
Payment processed
Order created
If an error occurs, the application may generate:
Database connection failed
We can check container logs using:
kubectl logs <pod-name>
In production, logs are often collected centrally using logging systems.
For example:
Pods
↓
Container Logs
↓
Log Collector
↓
Central Logging System
↓
Search / Analysis
Tools commonly used in Kubernetes logging architectures include Fluent Bit and other log collectors.

Simple flow:
Application → Container Logs → Log Collector → Central Logging → Search / Monitoring

Main point:
Kubernetes Logging = Collects and analyzes application and container logs.
;;;
60. What is Kubernetes Troubleshooting?
Kubernetes Troubleshooting is the process of identifying and fixing problems in Pods, Deployments, Services, Nodes, networking, storage, and other Kubernetes resources.
EXAMPLE:
Suppose our e-commerce backend Pod is showing:
CrashLoopBackOff
We first check the Pods:
kubectl get pods
Then we inspect the Pod:
kubectl describe pod <pod-name>
Then we check the application logs:
kubectl logs <pod-name>
We may find that the application is crashing because of:
Wrong configuration
Missing Secret
Database connection failure
Invalid image
Insufficient resources
Another common problem is:
ImagePullBackOff
This may indicate that Kubernetes cannot pull the specified container image.
For a Service problem, we may check:
kubectl get svc
kubectl describe svc <service-name>
kubectl get endpoints
For a Pending Pod, we can inspect:
kubectl describe pod <pod-name>
to look for scheduling, resource, affinity, taint, or other events.

Simple flow:
Problem → kubectl get → kubectl describe → kubectl logs → Check Events / Resources / Configuration → Find Root Cause → Fix Problem

Main point:
Kubernetes Troubleshooting = Identify the root cause of cluster/application problems and restore the workload to the desired state.
ChatGPT is AI and can make mistakes.
Chat with ChatGPT